Narzędzie wspierające wizualne tworzenie kodu dla mikrokontrolerów Arduino

(A tool supporting visual code creation for Arduino microcontrollers)

Dominik Baziuk, Jakub Mikołajczyk

Praca inżynierska
Promotor: dr Wiktor Zychla

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

Streszczenie

Opracowanie obejmuje stworzenie narzędzia wspierającego wizualne tworzenie kodu dla mikrokontrolerów Arduino. Przeanalizowano dostępne rozwiązania użyte w innych projektach rozpatrujących podobny problem. Zdecydowano się stworzyć aplikację za pomocą silniku Unity Engine. Przeanalizowano potrzeby użytkownika i stworzono koncept blokowy dostosowany do tworzenia kodu używanego dla mikrokontrolerów Arduino. Wymyślono prosty interfejs graficzny, aby nie obciążać nadmiernie uwagi użytkownika. Jakość stworzonej aplikacji przetestowano testami jednostkowymi i integracyjnymi za pomocą narzędzia „Unity Test Framework". Wygenerowano prosty kod i wgrano do mikrokontrolera Arduino.

Spis treści

1 Ważniejsze skróty i oznaczenia **3**

2 Wprowadzenie **3**

3 Opis i analiza zagadnienia **4**

4 Opis zastosowanych rozwiązań **4**
4.1 Przekrój podobnych rozwiązań . . . 4
4.2 Unity Engine . . . 4
4.3 Zastosowane rozwiązania . . . 5

5 Część dla użytkowników **5**
5.1 Instalacja programu . . . 5
5.2 Podręcznik użytkownika . . . 5
5.2.1 Informacje ogólne . . . 5
5.2.2 Widok Startowy . . . 5
5.2.3 Node Block . . . 6
5.2.4 Ruch kamerą . . . 8
5.2.5 Lista skrótów klawiszowych . . . 9
5.2.6 Lista funkcji użytkownika . . . 10
5.2.7 Lista zmiennych globalnych . . . 11
5.2.8 Lista zmiennych lokalnych . . . 11
5.2.9 Lista funkcji wbudowanych . . . 12
5.2.10 Edytor funkcji . . . 12
5.2.11 Edytor zmiennych . . . 13
5.2.12 Edytor Input(stałych) . . . 14
5.2.13 System Message . . . 15
5.2.14 MenuBar . . . 15

6 Część dla programistów **16**
6.1 Narzędzia użyte w projekcie . . . 16
6.2 Struktura projektu . . . 16
6.2.1 Architektura . . . 16
6.2.2 Backend . . . 17
6.2.3 Frontend . . . 21

7 Przypadki użycia **22**
7.1 Dodanie funkcji . . . 22
7.2 Edycja funkcji . . . 22
7.3 Dodanie zmiennej . . . 22
7.4 Edycja zmiennej . . . 23
7.5 Dodanie input node'a . . . 23
7.6 Połącznie bloków . . . 23
7.7 Prosty funkcja zwiększająca wartość o jeden . . . 23
7.8 Scenariusz zapisu projektu . . . 25
7.9 Scenariusz wczytania projektu . . . 25

8 Dodatkowe informacje **25**
8.1 Metryki kodu . . . 25
8.1.1 Frontend . . . 25
8.1.2 Backend . . . 25

9 Podsumowanie **25**
9.1 Wnioski . . . 25
9.2 Usprawnienia . . . 26

1 Ważniejsze skróty i oznaczenia

Node Block W tekście oznaczać będzie to definicje funkcji, zmiennej, stałej w postaci klasy.

Obiekt Node Block W tekście oznaczać będzie to reprezentacje Node Block'a w aplikacji jako obiekt z możliwością interakcji.

Unity Engine Silnik gier komputerowych.

Punkt połączenia W tekście to wizualna reprezentacja argumentów funkcji, wartości zwracanej przez funkcję lub zmienną, a także kolejności przepływu sterowania między kolejnymi operacjami.

Połączenie Node Block'ów W tekście będzie to oznaczenie na wizualną reprezentację połączenia między Node Block'ami za pomocą czarnej linii łączącej odpowiednie punkty połączeń.

Prefab (prefabrykat) Jest to matryca, na podstawie, której, Unity potrafi stworzyć dowolną ilość takich obiektów na scenie.

Scena miejsce, w którym pracujesz z treścią w silniku Unity.

Klatka obraz generowany w silnik, aby pokazać użytkownikowi aktualny stan aplikacji. Generowany i pokazywany co jakiś interwał czasu.

Pole wpisywalne Element UI, w którym użytkownik po naciśnięciu może wpisać za pomocą klawiatury jakiś tekst.

IO jest to reprezentacja punktu połączenia w backendzie.

Node jest to reprezentacja node block w backendzie.

flowNode jest to node, który umożliwia przepływ sterowania.

PathName jest to reprezentacja id za pomocą ścieżki.

2 Wprowadzenie

Przedmiotem pracy jest wytworzenie narzędzia wspierającego wizualne tworzenie kodu dla mikrokontrolerów Arduino. Chcąc zrozumieć idea wytworzenia takiego narzędzia trzeba wspomnieć o paru faktach opisanych w kolejnych akapitach.

Arduino – platforma programistyczna dla systemów wbudowanych oparta na prostym projekcie Open Hardware przeznaczonym dla mikrokontrolerów montowanych w pojedynczym obwodzie drukowanym, z wbudowaną obsługą układów wejścia/wyjścia oraz standaryzowanym językiem programowania.

Prosty zarówno koncept jak i interfejs platformy programistycznej Arduino poruszył duże grono ludzi niekoniecznie zaznajomionych w technikaliach mikrokontrolerów do tworzenia własnych projektów. Ludzie ci tworzyli rozmaite układy, aby pomóc sobie w codziennym życiu. Dobrym przykładem tego były projekty prostych systemów kontrolujących oświetleniem w mieszkaniu. Znajdą się również ludzie, który dali się ponieść kreatywności i użyli system Arduino do stworzenia własnych robotów niekoniecznie używanych do codziennych czynności domowych.

We współczesny oprogramowaniu, stronach internetowych, aplikacjach czy grach komputerowych można zauważyć ogromny wkład artystów w warstwe wizualną tych produktów. Wielu artystów chciałoby tworzyć takie projekty, ale nie mają zaawansowanej wiedzy w dziedzinie programowania, co jest kluczowe

do wytwarzania warstwy logicznej wspomnianych produktów. Najczęściej artyści znają chociaż podstawy programowania.

Nasz projekt opisany w tej dokumencie ma za zadanie wesprzeć artystów i dostarczyć im proste narzędzie dzięki, któremu będą mogli tworzyć warstwę logiczną swoich projektów na popularnych mikrokontrolerach Arduino.

3 Opis i analiza zagadnienia

Początkowy projekt aplikacji miał na celu stworzenie prostego narzędzia, które umożliwi użytkownikowi zarządzanie komponentami podłączonymi do płytki Arduino, takimi jak diody, serwomechanizmy czy przełączniki. Wkrótce jednak okazało się, że użytkownicy potrzebują funkcji znanych z języków programowania, aby tworzyć programy z dowolną funkcjonalnością. Użytkownik mógł poczuć się ograniczony gdyby jedynymi operacjami były te dedykowane dla komponentów.

Aby sprostać tym wymaganiom, zidentyfikowano kluczowe elementy języków programowania, takie jak funkcje (definiowane przez użytkownika jak i wbudowane), zmienne (z podziałem na globalne i lokalne) oraz stałe, które należało wprowadzić do aplikacji. W odpowiedzi na te potrzeby stworzono koncepcję "Node Blocków"– abstrakcyjnych struktur, umożliwiających pracę z tymi elementami w wizualnym języku programowania.

Następnie zaprojektowano interfejs użytkownika, wprowadzając listy, edytory i inne kontrolki, które pozwalają na intuicyjne zarządzanie Node Blockami. To podejście zostało szczegółowo opisane w rozdziale „Podręcznik użytkownika", co uczyniło aplikację bardziej kompleksowym narzędziem.

Ważnym aspektem tego projektu było także odpowiednie przypisanie odpowiedzialności każdemu z autorów. W ramach tego podziału aplikacja została rozdzielona na dwie główne części: Backend i Frontend. Taki podział umożliwił bardziej efektywną pracę, ale jednocześnie wymagał stałej i ścisłej komunikacji między autorami szczególnie w procesach projektowania i implementacji systemu.

4 Opis zastosowanych rozwiązań

4.1 Przekrój podobnych rozwiązań

Unreal Engine Unreal Engine – silnik gier komputerowych, wyprodukowany przez przedsiębiorstwo Epic Games. Pierwotnie wykorzystywany w strzelankach pierwszoosobowych, z czasem zaczął być wykorzystywany również w innych gatunkach gier komputerowych. Oferuje on system Blueprints wspierający wirtualne tworzenie kodu. Ważną funkcjonalnością jest możliwość optymalizacji wygenerowanego kodu po przez dopisywanie własnych modułów w językiem programowania c++.

Visuino Visuino - generator kodu, który używa Arduino IDE do kompilacji i wgrywania programów. Główną różnicą co do naszego rozwiąnia jest to, że Visuino opiera się na budowie układów elektrycznych i bloczki używane tam odpowiadają rzeczywistej reprezentacji płytki. Natomiast idea bardziej skupia się na semantyce języków programowania, stąd w późniejszych rozdziałach można zobaczyć podział na funkcję, zmienne i stałe, a także widoczny przepływ sterowania między bloczkami (tak jak wykonywania się kodu instrukcja po instrukcji).

4.2 Unity Engine

Unity Engine jest to silnik do tworzenia trójwymiarowych oraz dwuwymiarowych gier komputerowych lub innych materiałów interaktywnych, takich jak wizualizacje czy animacje. Działa na systemie operacyjnym Microsoft Windows, macOS oraz Linux i pozwala tworzyć aplikacje na przeglądarki internetowe, komputery osobiste, konsole gier wideo oraz urządzenia mobilne. Pierwszy raz zostało zaprezentowane na konferencji Worldwide Developers Conference w 2005 roku

4.3 Zastosowane rozwiązania

Unity Engine zastosowany w naszym rozwiązaniu oferuje przejrzysty i efektywny edytor. Sposób tworzenia obiektów na scenie z jego pomocą jest prosty i zrozumiały. Zdecydowano się na jego użycie podczas tworzenia tej aplikacji ze względu na duże doświadczenie autorów pracy dyplomowej w korzystaniu z silnika.

5 Część dla użytkowników

5.1 Instalacja programu

Node-Base-Arduino-IDE mieści się w repozytorium na GitHubie [1]. Należy mieć zainstalowany Unity Engine w wersji minimalnie 2022.3.7f1. Pobieramy projekt z repozytorium i otwieramy go w edytorze Unity Engine. Użytkownik może następnie wygenerować Build, czyli plik wykonywalny dla systemu operacyjnego z którego aktualnie korzysta (Unity Engine pozwala na wygenerowanie ich dla między innymi systemu Windows, Mac OS i Linux). Po włączeniu IDE z wygenerowanego Builda powinno się pokazać okno aplikacji, a pierwsze co zobaczy użytkownik to logo Unity Engine, a następnie widok startowy opisany w następnej podsekcji.

5.2 Podręcznik użytkownika

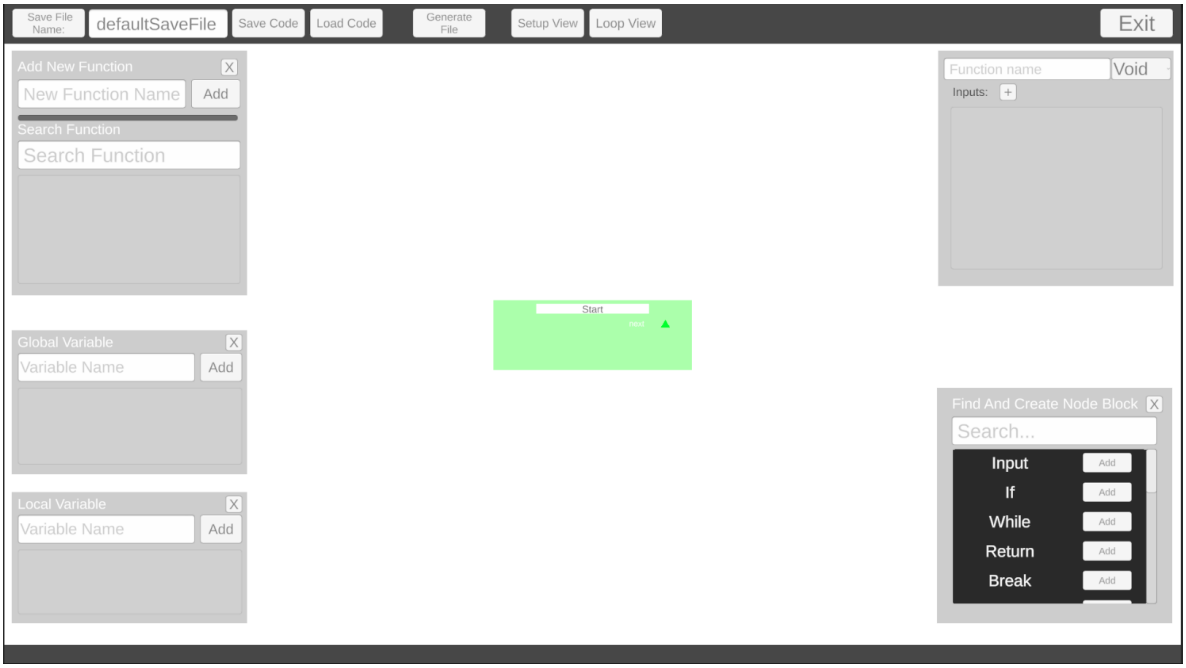
5.2.1 Informacje ogólne

Node Base Arduino IDE pozwala na szereg operacji pozwalających na zarządzanie odpowiednimi strukturami języka programowania wedle uznania użytkownika. W następnych podsekcjach znajdują się opisy poszczególnych widoków i edytorów niezbędnych do tego, aby można było wygenerować użyteczny kod.

Warto zaznaczyć, że stan edytora nie jest zapisywany automatycznie i użytkownik samemu decyduje kiedy zapisać zmiany lub z jakiego pliku wczytać stan (więcej o tym w podsekcji Menu)

5.2.2 Widok Startowy

Po włączeniu aplikacji powinnien się ukazać widok startowy (tak jak na obrazku 7). W tym momencie użytkownik powinnien być na scenie w ciele funkcji „loop”, gdzie jedynym elementem jest Node Block startowy, od którego zaczyna się przepływ kodu.



Rysunek 1: Widok Startowy, ciało funkcji „loop”

5.2.3 Node Block

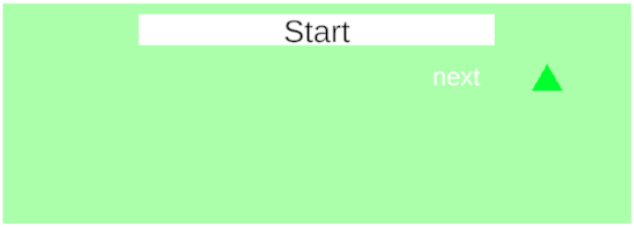
Node Block to element na scenie w postaci prostokąta z nazwą i odpowienimi połączeniami reprezentującym obiekty w programowaniu takie jak funkcje, zmienne czy stałe. W związku z różnorodnością takich obietków zdecydowaliśmy się na następujące rodzaje Node Blocków.

Funkcje
Node Block reprezentujący funkcję. Posiada nazwę funkcji, a także połączenia w zależności od argumentów funkcji i zwracanej wartości. Gdy funkcja posiada typ zwracany "Void"do jej reprezentacji dodawane są połączenia next i prev związane z przepływem kodu.

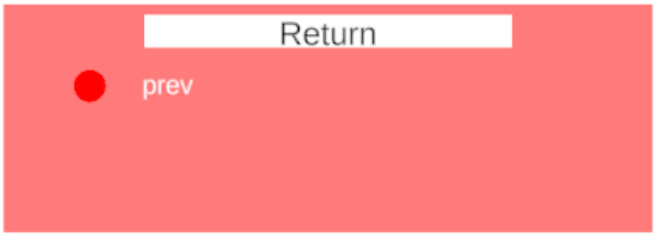


Rysunek 2: Przykładowy Node Block funkcji

Node Block Start i Return
Dwa Node Bloki zaznaczające początek i koniec przepływu funkcji, której ciało aktualnie edytujemy. Node Block Start oznaczony jest kolorem zielonym i ma jedynie punkt połączenia next. Natomiast Node Block Return, oprócz punktu połączenia prev może mieć jeszcze punkt połączenia związany z wartością i jej typem zwracanym przez funkcje.

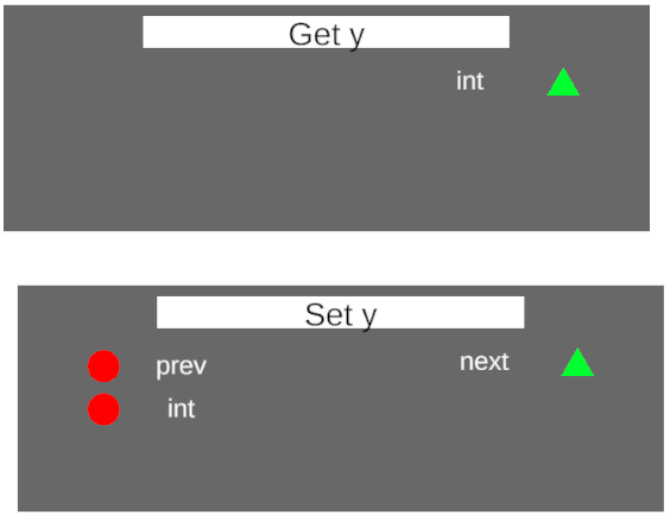


Rysunek 3: Przykładowy Node Block Start



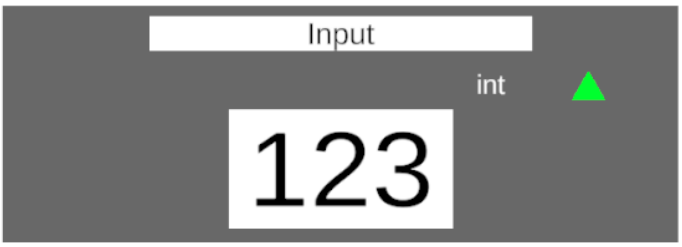
Rysunek 4: Przykładowy Node Block Return

Zmienne
Zmienne reprezentowane są przez dwa Node Bloki get i set. Pierwszy z nich ma tylko połączenie odpowiadające wartości i typowi zmiennej. Natomiast ten drugi odpowiedzialny jest za ustawianie wartości zmiennej i posiada dodatkowo połączenia odpowiadające za przepływ kodu.



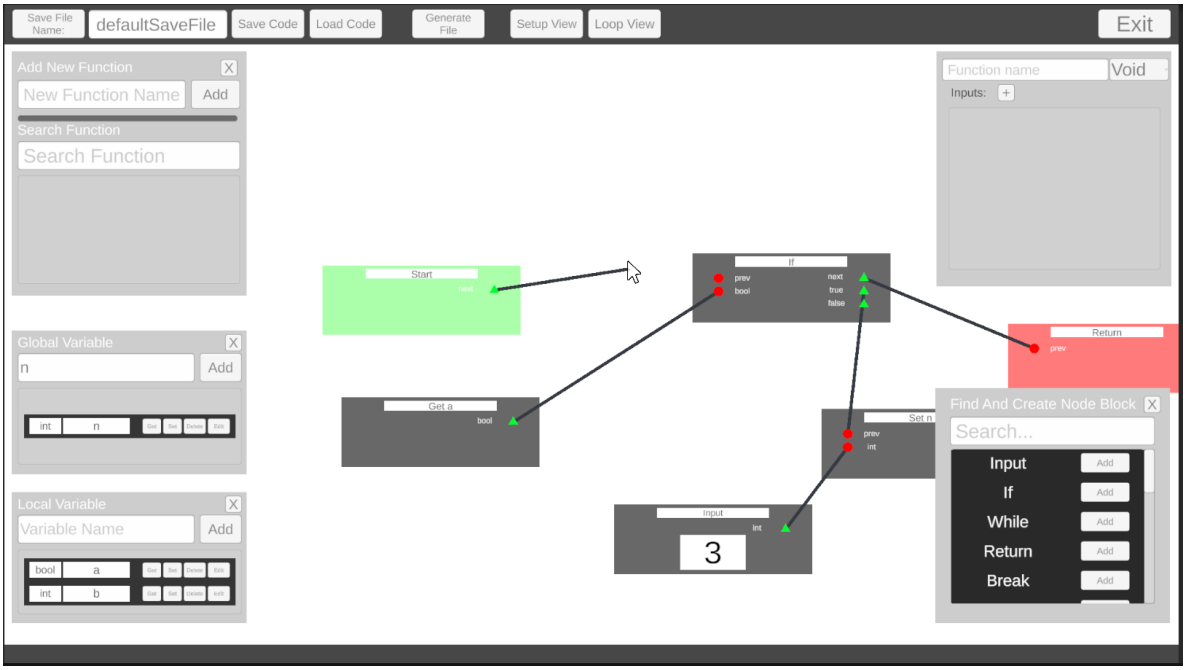
Rysunek 5: Oba warianty Node Block'u dla zmiennej

Input(stałe)
Specjalny Node Block z polem nazwy ustawionym na Input i polem pokazującym jego aktualna wartość. Można mu podać odpowiednią wartość i odpowiedni typ. Reprezentuje on stałe w kodzie.



Rysunek 6: Przykładowy Node Block Input z ustaloną wartością

Połączenia
Użytkownik może połączyć odpowiednie Node Block'i. Aby to zrobić naciska na jeden z punktów połączenia lewym przyciskiem (przytrzymując ten przycisk) , przeciąga w miejsce drugiego punktu połączenia i wtedy zwalnia przycisk myszy. Jeśli typy punktów połączeń odpowiadały zasadą ich łączenia to czarna linia między nimi będzie symbolizować, że są aktualnie połączone. Aby usunąć połączenie należy na jeden z punktów połączeń najechać kursorem myszy, przycisnąć lewy przycisk razem z ctrl. Czarna linia powinna zniknąć symbolizując rozłączenie Node Block'ów.

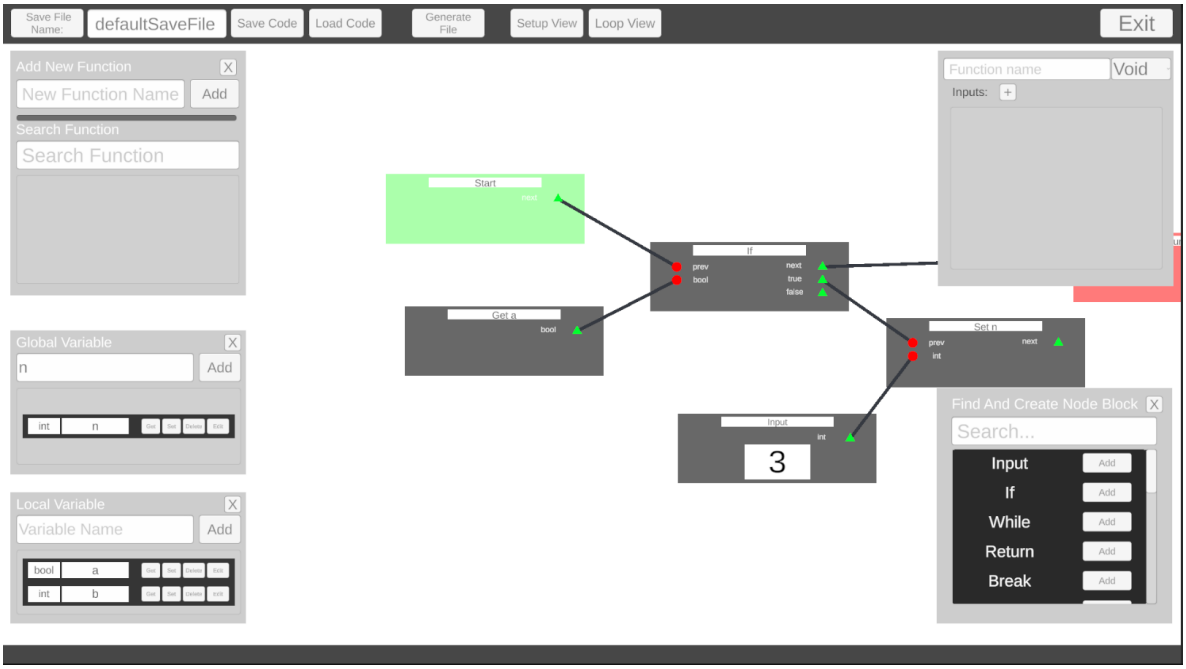


Rysunek 7: Kliknięcie punktu połączenia i przytrzymanie wyświetla za kursorem czarną linię

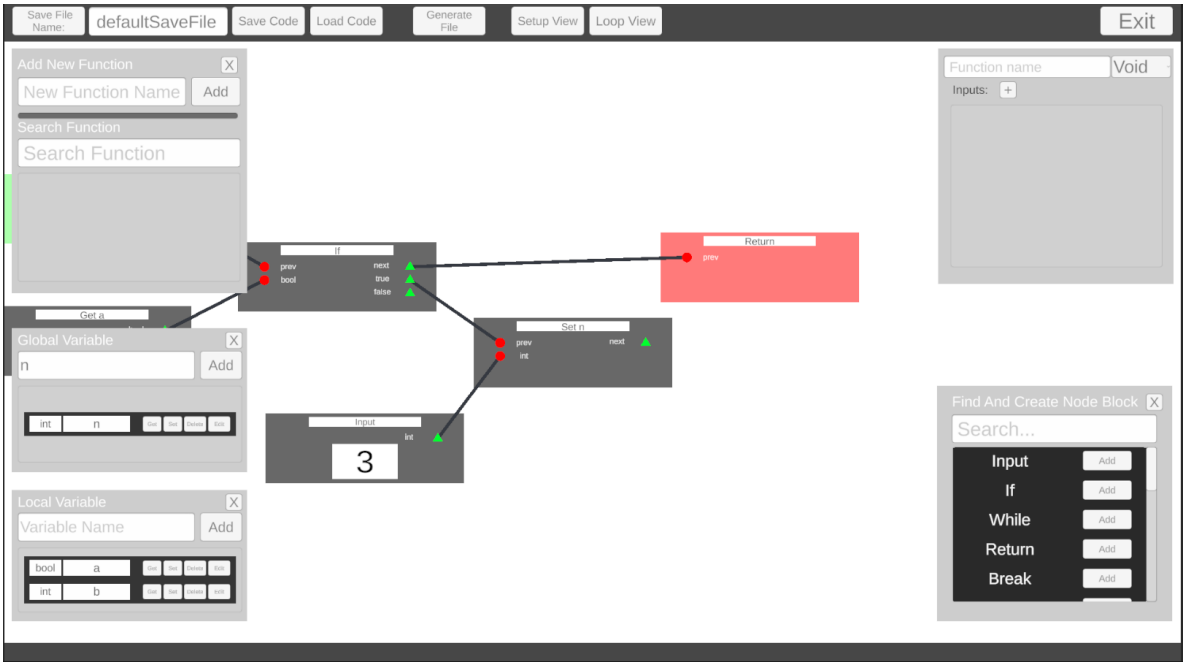
5.2.4 Ruch kamerą

Podczas korzystania z IDE można wykonywać ruch kamerą. Należy nacisnąć ctrl + prawy przycisk myszy na białe tło edytora kursorem. Dopóki trzymana jest kombinacja przycisków można dowolnie poruszać kamerą po scenie.

Drugą operacją ułatwiającą korzystanie z aplikacji jest możliwość przybliżania i oddalania. Wystarczy użyć scrolla myszki.



Rysunek 8: Stan edytora przed ruchem kamerą w bok



Rysunek 9: Stan edytora po ruchu kamerą w bok

5.2.5 Lista skrótów klawiszowych

Aplikacja posiada znaczną ilość skrótów klawiszowych ułatwiających korzystanie z aplikacji. W poprzednich sekcjach była mowa o paru. Poniżej zamieszczono resztę dostępnych akcji:

- Przesunięcie Node Block'a - naciśnięcie i przytrzymanie lewego przycisku myszy na Node Block'u, następnie przesunięcie kursora w wybrane miejsce.

- Usunięcie Node Block'a - naciśnięcie lewy ctrl + lewy przycisk myszy na Node Block'u
- Połączenie Node Block'a z innym - naciśnięcie i przytrzymanie lewego przycisku myszy na punkcie połączenia następnie przesunięcie kursora nad drugi punkt połączenia i puszczenie lewego przycisku myszy.
- Ruch kamerą - naciśnięcie i przytrzymanie lewy ctrl + prawy przycisk myszy na białym tle
- Przybliżanie widoku kamery - użycie scrolla myszy.
- Włączenie Listy funkcji użytkownika - naciśnięcie klawiszu „F"
- Włączenie Listy zmiennych globalnych - naciśnięcie klawiszu „G"
- Włączenie Listy zmiennych lokalnych - naciśnięcie klawiszu „L"
- Włączenie Listy funkcji wbudowanych - naciśnięcie klawiszu „D"
- Włączenie Edytora Input(stałych) - naciśnięcie lewy shift + lewy przycisk myszy na Node Block Input.

5.2.6 Lista funkcji użytkownika

Lista funkcji użytkownika jest kontrolką, która pozwala na dodawanie własnych funkcji przez użytkownika jak i wybieranie już dodanych.

Dodać nową funkcję można po przez podanie jej nazwy w pierwszym polu wpisywania. Naciśnięcie przycisku „Add" spowoduje dodanie jej do listy.

Drugie pole wpisywania umożliwia wyszukanie odpowiednich funkcji po przez wpisanie całości lub części ich nazwy.

Lista funkcji zawiera obiekty zwane w kodzie FunctionButton. Posiadają pole z nazwą funkcji oraz cztery przycisk:

- Add - powodujący pojawienie się Node Block'a z funkcją na scenie
- Delete - usunięcie funkcji z listy jak i wszystkich Node Block'ów tej funkcji z poszczególnych implementacji innych funkcji
- Edit - rozpoczęcie edycji funkcji w Edytor funkcji
- Change - przełączenie aktualnego widoku aplikacji na impementacje funkcji.



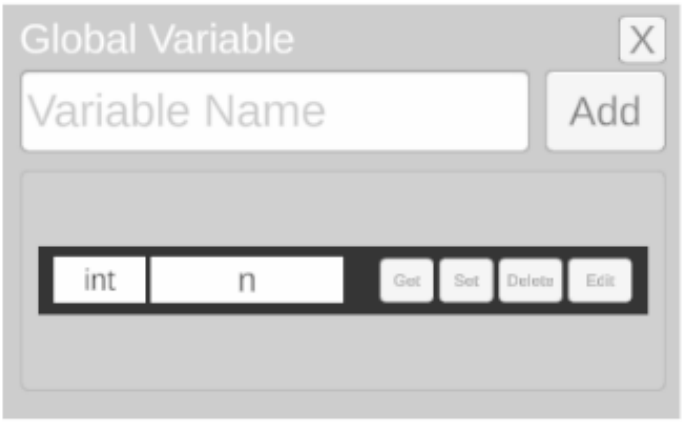
Rysunek 10: Przykładowy stan listy funkcji użytkownika

5.2.7 Lista zmiennych globalnych

Lista zmiennych globalnych jest kontrolką, która pozwala na dodawanie zmiennych globalnych. Pole wpisywania ma dwa zastosowania, jednocześnie służy do dodawania nowych zmiennych po przez naciśnięcie przycisku „Add" jak i filtrowania listy zmiennych.

Na liście zmiennych globalnych znajdują się obiekty zwane w kodzie VariableButton. Posiadają pole z nazwą funkcji oraz cztery przycisk:

- Get - Dodający Node Block Get zmiennej do aktualnego widoku (implementacji funkcji)
- Set - Dodający Node Block Set zmiennej do aktualnego widoku (implementacji funkcji)
- Edit - rozpoczęcie edycji zmiennej w Edytor zmiennych
- Delete - usunięcie zmiennej z listy jak i wszystkich Node Block'ów tej zmiennej z poszczególnych implementacji innych funkcji



Rysunek 11: Przykładowy stan listy zmiennych globalnych

5.2.8 Lista zmiennych lokalnych

Ma taką samą funkcjonalność jak lista zmiennych globalnych. Różnica polega na tym, że zawartość jej zmienia się wraz z przełączaniem widoków funkcji (implementacji funkcji).

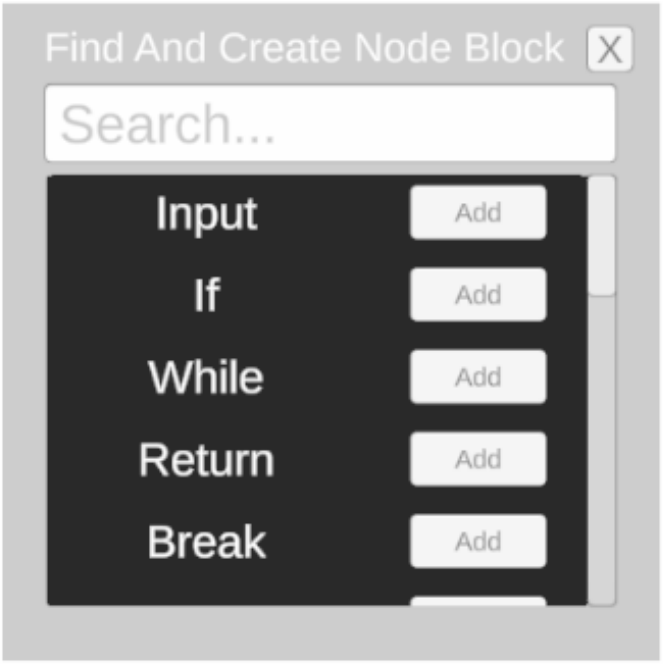


Rysunek 12: Przykładowy stan listy zmiennych lokalnych

5.2.9 Lista funkcji wbudowanych

Lista funkcji wbudowanych posiada funkcję niezbędne do tworzenia kodu. Funkcje te są wczytywane z plików json. Pole wpisywania służy do filtrowania listy funkcji wbudowanych.

Na liście znajdują się obiekty zwane ReferenceButton. Odpowiadają one dodanym przez nas funkcją wbudowanym. Posiadają pole z nazwą i przycisk „Add"pozwalający na dodanie Node Block'u z funkcją wbudowaną do aktualnego widoków funkcji (implementacji funkcji).



Rysunek 13: Przykładowy stan listy funkcji wbudowanych

5.2.10 Edytor funkcji

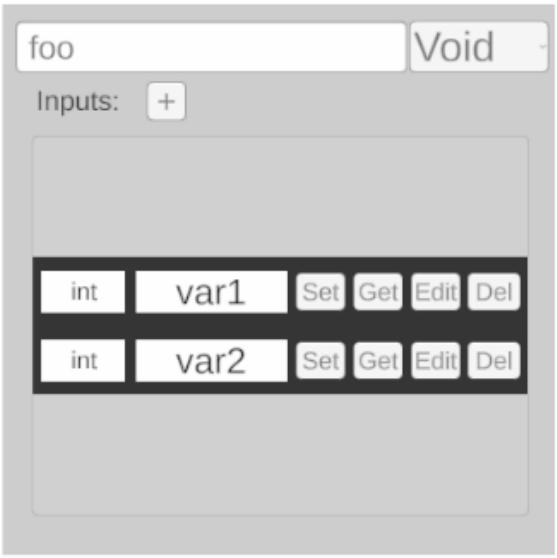
Edytor funkcji posiada następujące trzy funkcjonalności.

Edycja nazwy funkcji
Użytkownik może zmienić nazwę funkcji po przez wpisanie nowej nazwy w pole wpisywanlne. Zmiana nazwy następuje automatycznie po zakończeniu wpisywania nowej nazwy.

Kontrolka dropdown
Kontrolka dropdown pokazuje po kliknięciu w nią listę możliwych typów dla wartości zwracanej funkcji. Wybranie którejś z opcji spowoduje zmianę typu zwracanego.

Przycisk „+"
Naciśnięcie go powoduje dodanie kolejnego argumentu funkcji.

Lista argumentów funkcji
Posiadą podobną funkcjonalność zarządzania argumentami w postaci zmiennych jak lista zmiennych lokalnych.



Rysunek 14: Przykładowy stan edytora funkcji

5.2.11 Edytor zmiennych

Edytor zmiennych to obiekt wyświetlany na scenie po wciśnięciu, któregoś z przycisków „Edit" dla zmiennych lokalnych, zmiennych globalnych jak i argumentów funkcji. Kontrolka ta posiada następujące funkcjonalności.

Edycja nazwy zmiennej
Nową nazwę należy wpisać w pole wpisywalne

Kontrolka dropdown
Kontrolka dropdown pokazuje po kliknięciu w nią listę możliwych typów zmiennej.

Przycisk „Save"
Spowoduje zapisanie zmian oraz aktualizacje Node Block'ów edytowanej zmiennej tam gdzie została użyta i zamyka edytor zmiennych.

Przycisk „Cancel"
Anuluje operację zmiany definicji zmiennej i zamyka edytor zmiennych.



Rysunek 15: Przykładowy stan edytora zmiennych

5.2.12 Edytor Input(stałych)

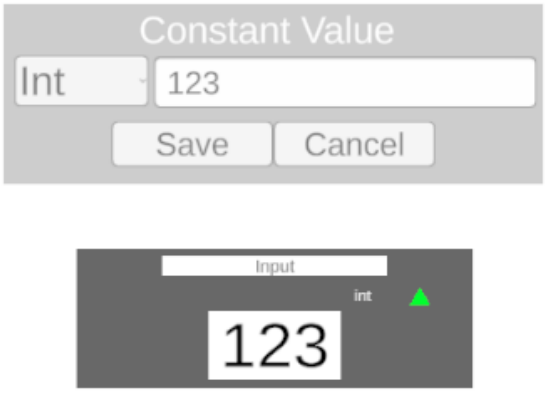
Edytor zmiennych to obiekt wyświetlany na scenie po wciśnięciu za pomocą Shift + lewy przycisk myszy kursurem, na któregoś z Node Blocków Input. Kontrolka ta posiada następujące funkcjonalności.

Edycja wartości stałej
Nową wartość należy wpisać w pole wpisywalne

Kontrolka dropdown
Kontrolka dropdown pokazuje po kliknięciu w nią listę możliwych typów stałej.

Przycisk „Save”
Spowoduje zapisanie zmian oraz aktualizacje Node Block Input edytowanego aktualnie i zamyka edytor Input(stałych).

Przycisk „Cancel”
Anuluje operację zmiany i zamyka edytor Input(stałych).

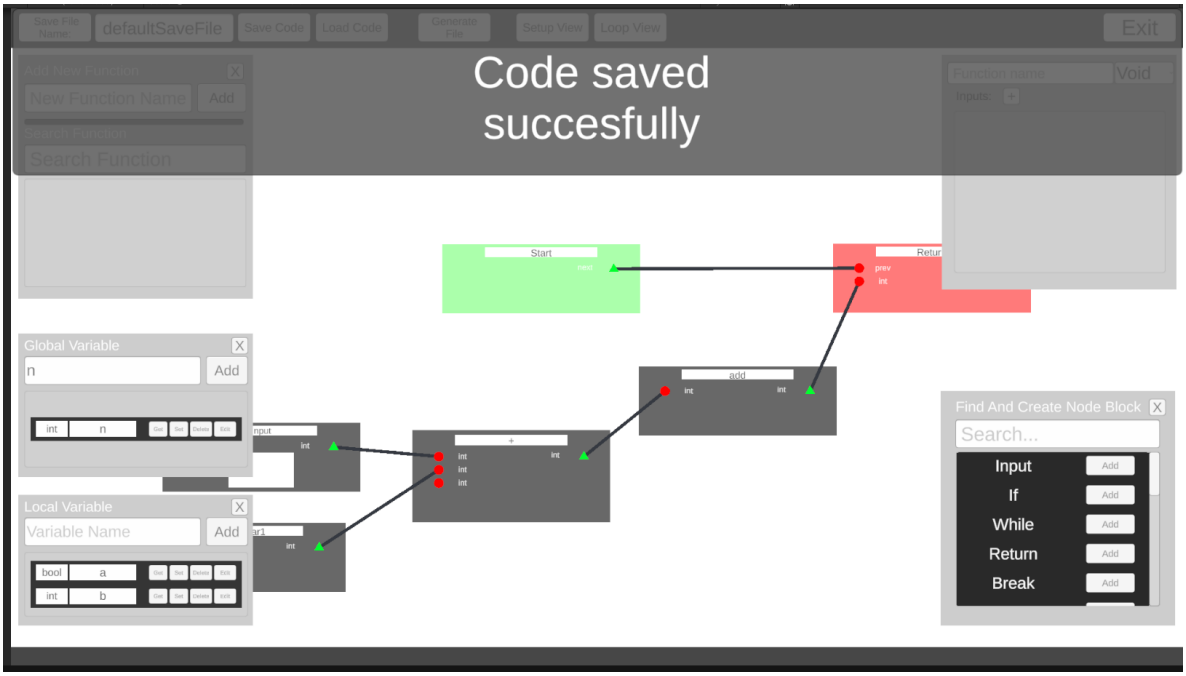


Rysunek 16: Przykładowy stan edytora Input(stałych)

5.2.13 System Message

System Message pozwala na komunikację naszej aplikacji z użytkownika w razie wykonywania niepoprawnych operacji.

System Message generuje obiekty w postaci szarych prostokątów i napisem z treścią błędu lub informacją o powodzeniu operacji. Po chwili obiekt ten zanika (efekt fade out).



Rysunek 17: Przykładowy informacji o powodzeniu operacji załadowania stanu z pliku

5.2.14 MenuBar

W MenuBar znajdują się następujące elementów.

Pole wpisywalne z nazwą pliku
Pole to służy do podania nazwy pliku, z którego będą korzystać następne przyciski.

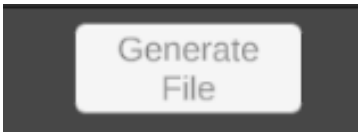
Przycisk „Save State”
Zapisuje stan edytora do pliku, którego nazwa podana jest w polu wpisywalnym.

Przycisk „Load State”
Ładuje stan edytora z pliku, którego nazwa podana jest w polu wpisywalnym.



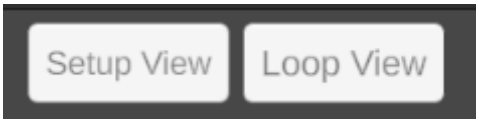
Rysunek 18: Grupa przycisków od zapisu do pliku

Przycisk generacji kod Zapisuje wygenerowany kod na podstawie stanu aplikacji do pliku „code.in”.



Rysunek 19: Przycisk generacji kodu

Przyciski widoku setup i loop
Odpowiednio przełączają widok funkcji (implementacje) między funkcjami kluczowymi dla kodu wykonywalnego arduino, czyli setup i loop.



Rysunek 20: Przyciski zmiany widoku funkcji między „setup" i „loop"

Przycisk „exit"
Wyłącza aplikacje



Rysunek 21: Przycisk wyjścia z aplikacji

6 Część dla programistów

6.1 Narzędzia użyte w projekcie

Aplikacja Node Base Arduino IDE była tworzona w silniku Unity Engine. Kod dla poszczególnych elementów sceny 2D (frontend) był w Visual Studio 2022 z rozszerzeniem „Visual Studio Tools for Unity". Natomiast kod backendu został napisany przy pomocy narzędzia JetBrains Rider 2023.3.4. Kod jest przechowywany w zdalnym repozytorium na githubie, a sama kontrola wersji odbywała się przy pomocy gita.

Aplikacja została uruchomiona na systemie Windows 10 oraz macOS M1. Backend został przetestowany za pomocą testów jednostkowych wykorzystana została do tego biblioteka NUnit, a także NSubstitute. Aplikacja została przetestowana również przy pomocy testów integracyjnych napisanych za pomocą „Unity Test Framework". Dla testów integracyjnych zostały stworzone odpowiednie sceny dla testowania poszczególnych kontrolek.

6.2 Struktura projektu

6.2.1 Architektura

Tworzenie aplikacji w silniku Unity Engine wymusiło podzielenie architektury na dwie części. Pierwsza z nich Frontend operuje na scenie i prefab'ach (obiekty na scenie, do których podpina się różne skrypty do osiągnięcia wymaganych funkcjonalności). Tworzy ona obiektowe reprezentacje Node Block'ów na scenie. Druga część, czyli Backend definiuje Node Blocki, zarządza ich częścią abstrakcyjną, a następnie pozwala na wygenerowanie kodu z powstałych struktur. Komunikajca z backend odbywa się za pomocą interfejsów oraz DTO (data transfer object).

6.2.2 Backend

PathName
Jest to klasa pomocnicza. Umożliwia ona otrzymanie nazwy aktualnej ścieżki oraz jej id. Pomaga ona także w otrzymaniu następnej ścieżki.

BackendManager
Jest to główna klasa służąca do komunikacji z backendem. Przechowuje ona wszystkie potrzebne informacje do zarządzania backendem. Takie jak zmienne globalne, funkcje setup i loop, a także funckje zdefiniowane przez użytkownika oraz wbudowane. Posiada możliwość zapisu/odczytu stanu do/z json. Odpowiedzialna jest ona także za budowanie kolejnych fragmentów kodu, poprzez przekazywanie CodeManagera. Następnie wygenorowany kod zostaje zapisany w pliku.

CodeManager
Klasa ta służy do stopniowego budowania kodu przechowywania statusu zmiennych oraz bibliotek. Kod jest stopniowo budowany poprzez dodawianie bibliotek, oraz kolejnych linii kodu. Ostateczny kod zostaje wygenerowany poprzez dodanie na początku bloku komentarza, następnie bibliotek z pominięciem ignorowanych, a na końcu kolejnych linni kodu. Klasa dodatkowo posiada konstruktor kopiujący, który kopiuje statusy zmiennych do nowej kolekcji oraz tworzy nową pustą kolekcję na linie kodu.

PrimitiveType, StringType, VoidType, ClassType
Są to klasy przechowujące typ. Implementują one dodatkowo metodę umożliwiającą sprawdzenie czy dwa dane typy są pomiędzy sobą castowalne.

BaseIO
Jest to abstrakcyjna klasa, służąca do zarządzania połączeniami. Przechowuje ona podstawowe informacje o IO takie jak nazwa, typ, rodzic (node), strona (wejście/wyjście). Podczas łączenia IO wyjście zawsze jest łączony z wejściem. Samo łączenie odbywa się w kilku etapach, które są kontrolowane za pomocą handlerów. Jednym z etapów jest precheck w którym następuje sprawdzanie, czy IO jest już połączone, strony są różne, rodzice są różni, nie powstał cykl. Jednym z kolejnych etapów jest już właściwy check. Klasa również umożliwia ponowną weryfikację w postaci recheck, podczas którego pomijany jest już precheck. W przypadku niepowodzenia rechecku następuje rozłączenie połączenia, które również przebiega w kilku etapach, kontrolowanych za pomocą hanlderów. W momencie usunięcia IO następuje rozłączenie, a następnie IO zostaje oznaczone jako logicznie usunięte.

FlowIO
Jest to klasa dziedzicząca po **BaseIO**. Służy ona do zarządzania specjalnym połączeniem jakim jest flow programu. Podczas łączenia w prechecku zostaje dodatkowo sprawdzone czy druga strona połączenia jest także typu FlowIO.

TypeIO
Jest to klasa dziedzicząca po **BaseIO**. Służy ona do zarządzanie połączeniem, które może przyjmować typ. Podobnie jak w FlowIO podczas precheck zostaje sprawdzone czy druga strona połączenia jest także typu TypeIO. Dodatkowo w check zostaje sprawdzone czy dane typy IO mogą zostać połączone. Klasa ta dodatkowo umożliwia zmianę typu IO, po którym następuje recheck. Klasa dodatkowo umożliwia subskrypcje na takie zdarzenia jak podłączenie, rozłączenie, zmiana typu IO.

AutoIO
Jest to klasa dziedzicząca po **TypeIO**. W przeciwienstwie do TypeIO nie ma na początku zdefiniowanego typu, jest on przyjmowany bezpośrednio przed podłączeniem drugiej strony. Natomiast przy rozłączaniu zmieniany jest z powrotem na typ niezdefiniowany. Dodatkowo istnieje możliwość ustawienia na stałe konkretnego typu, a także późniejsze zresetowanie go.

ChainIO
Jest to klasa dziedzicząca po **AutoIO**. Przechowuje ona referencje do poprzedniego i następnego IO (można o tym myśleć jak o liście dwukierunkowej, gdzie IO jest konkretnym węzłem). Podobnie jak w AutoIO podczas podłączenia zostaje zdefiniowany typ, który następnie jest propagowany do końca listy. Natomiast podczas rozłączania typ jest resetowany dopóki na liście nie zostanie znaleziony podłączony IO. Klasa posiada również flagę dzięki której można ustawić, czy lista ma się sama rozszerzać (został podłączony ostatni IO) lub skracać (został rozłączony inny IO niż pierwszy lub pierwszy wejściowy). Dodatkowo można ustawić na stałe typ jaki mają mieć wszystkie IO na liście.

BaseNode
Jest to klasa abstrakcyjna służąca do zarządzania nodem. Przechowuje ona typ, nazwę noda, a także id creatora dzięki któremu można odtworzyć danego noda. IO są przechowywanie w liście z podziałem na wejścia i wyjścia, listy te są wypełniane przez klasy dziedziczące. Klasa ta dodatkowo przechowuje i tworzy podstawe flowIO jakimi są prev i next. Te IO mogą zostać dodane na liste przy pomocy metody pomocniczej, gdy klasa dziedzicząca jest flowNodem. Sama klasa posiada wiele metod pomocniczych, kolejną taką metodą może być usuwnie z listy prev i next. Klasa ta implementuje generacje kodu, na początku wykonywane jest sprawdzenie czy wszystkie wymagane IO są połączone. Następnie jeśli jest to zwykły node kod zostaje zwrócony w postaci stringa. Natomiast jeśli jest to flowNode kod zostaje dodany do codeManagera i następuje generacja kodu dla następnego noda (next). W momencie gdy node jest usuwany najpierw zostają usunięte wszystkie IO, a następnie node zostaje oznaczony jako logicznie usunięty.

BuildInNode
Jest to klasa dziedzicząca po **BaseNode**. Przechowuje ona dodatkowo template. Dzięki któremu można uzyskać nazwę node oraz jego id.

IfNode
Jest to klasa dziedzicząca po **BuildInNode**. Jest to flowNode umożliwiający stworzenie instrukcji warunkowej if, który przechowuje dodatkowo bool typeIO, który jest predykatem oraz flowIO dla ścieżki true oraz opcjonalne dla false. Podczas generacji kodu najpierw zostaje wygenerowany kod dla ścieżki true, a następnie dla false w zależności czy został podłączony, przed generacją kodu dla poszczególnej ścieżki zostaje skopiowany codeManager.

WhileNode
Jest to klasa dziedzicząca po **BuildInNode**. Jest to flowNode umożliwiający stworzenie pętli while, który przechowuje dodatkowo bool typeIO, który jest predykatem oraz flowIO dla ścieżki loop, która jest ciałem pętli. Generacja kodu odbywa się podobnie jak w ifNode.

ReturnNode
Jest to klasa dziedzicząca po **BuildInNode**. Jest to flowNode rozpoczynający przepływ, co za tym idzie nie posiada on wejścia (prev).

ReturnNode
Jest to klasa dziedzicząca po **BuildInNode**. Jest to flowNode kończący przepływ, zatem nie posiada on wyjścia (next). W zależności od funckji w jakiej się znajduje przechowuje typeIO odpowiedniego typu lub null jeśli funkcja jest typu void. Typ ten można zmienić. W momencie usunięcia node, dodatkowo następuje usunięcie referencji w funkcji.

InputNode
Jest to klasa dziedzicząca po **Build**. Klasa ta umożliwia zadeklarowanie wartości. Przechowuje ona dodatkowo wartość w postaci stringa. Wyjściem jest typeIO, którego domyślnym typem jest int. Typ ten oraz wartość może zostać zmieniona poprzez wywołanie odpowiedniej metody, podczas której następuje walidacja poprawności. Jeślli w momencie generowania kodu wartość nie została ustawiona zostaje użyta domyślna wartość dla danego typu.

ArithmeticOpNode
Jest to klasa dziedzicząca po **BuildInNode**. Jest to zwykły node umozliwiający operacje arytmetyczne, którego wyjście oraz wejścia są typu chainIO bez zdefiniowanego typu z automatycznym rozszerzanie. Wyjście jest głową listy.

LogicalOpNode
Jest to klasa dziedzicząca po **BuildInNode**. Jest to zwykły node umożliwiający operacje logiczne, którego wyjście jest bool typeIO. Natomiast wejście jest typu chainIO ze zdefiniowanym typem bool oraz możliwością rozszerzania.

CompareNode
Jest to klasa dziedzicząca po **BuildInNode**. Jest to zwykły node umożliwiający operację porównanie, które wyjście jest bool typeIO. Natomiast 2 wejścia są typu chainIO bez zdefiniowanego typu i bez możliwości rozszerzania.

VariableNode
Jest to klasa dziedzicząca po **BaseNode**. Przechowuje ona dodatkowo informacje o zmiennej oraz typeIO, który jest tego samego typu co zmienna. Klasa ta umożliwia zmiane typu.

SetVariableNode
Jest to klasa dziedzicząca po **VariableNode**. Jest to flowNode, umożliwiający ustawienie wartości zmiennej. Podczas generowania kodu w codeManager zostaje ustawiony status zmiennej.

GetVariableNode
Jest to klasa dziedzicząca po **VariableNode**. Jest to zwykły node, umożliwiający użycie zmiennej. Podczas generowania zostaje sprawdzone status zmiennej, czy jest globalna, parametrem lub została ustawiona przez setter.

FunctionNode
Jest to klasa dziedzicząca po **BaseNode**. Klasa ta umożliwia wywołanie funkcji wbudowanej. Przechowuje ona dodatkowo template funkcji na podstawie, którego generowane jest wejście oraz wyjścia. Jeśli funkcja jest typu void to jest ona flowNode, w przeciwnym razie jest zwykłym nodem. Podczas generowania kodu do codeManagera zostaje dodana biblioteka z której pochodzi funkcja.

ClassMethodNode
Jest to klasa dziedzicząca po **FunctionNode**. Pierwszym wejściem innym niż flowIO jest typeIO, którego typem jest klasa dla której zostanie wywołana metoda.

ClassConstructorNode
Jest to klasa dziedzicząca po **FunctionNode**. Pierwszym wejściem innym niż flowIO jest typeIO, którego typem jest klasa dla którego zostanie wywołany konstruktor.

UserFunctionNode
Jest to klasa dziedzicząca po **BaseNode**. Klasa ta umożliwia wywołanie funkcji stworzonej przez użytkownika. Przechowuje ona dodatkowo funkcje, na podstawie której generowane jest wyjście oraz wejścia. Jeśli funkcja jest typu void to jest ona flowNode, w przeciwnym razie jest zwykłym nodem. Umożliwia ona dodatkow dodawanie nowych wejść. A także zmianę typu wyjścia.

Variable
Klasa ta służy do zarządzania konkretną zmienną. Przechowuje ona referencje wszystkich nodów, które są instancjami typu node danej zmiennej. Dzięki czemu podczas zmiany nazwy/typu zmiennej, następuje aktualizacja odpowiednich nodów. Podczas zmiany nazwy następuje walidacja. Umożliwia ona także tworzenie nowych instacji node typu get/set.

VariablesManager
Jest to abstrakcyjna klasa służąca do zarządzania zmiennymi. Pozwalana ona na dodanie nowej zmiennej, oraz usunięcie. Podczas dodawania następuje walidacja unikalności nazwy zmiennej za pomocą abstrakcyjnej metody. Umożliwia ona również znalezienie konkretnej zmiennej za pomocą PathName.

LocalVariablesManager
Jest to klasa dziedzicząca po **VariablesManager**. Służy ona do zarządzania lokalnymi zmiennymi w danej funckji.

ParamsManager
Jest to klasa dziedzicząca po **VariablesManager**. Służy ona do zarządzania parametrami funckji. Posiada ona referencje do funckji użytkownika. Dzięki czemu podczas dodawania/usuwania zmiennej (parametru) funckja zostaje powiadomiona o zmianie.

GlobalVariablesManager
Jest to klasa dziedzicząca po **VariablesManager**. Służy ona do zarządzania globalnymi zmiennymi. Unikalność nazwy zmiennej jest sprawdzana we wszystkich funkcjach.

Function
Jest to klasa służąca do zarządzanie funkcjami. Przechowuje ona informacje o nazwie funkcji, typie wyjściowym, pathname, startNode, lokalnych zmiennych (LocalVariablesManager) oraz referencje do returnNodów znajdujących się w danej funkcji. Dodatkowo umożliwia sprawdzenie czy zmienna jest lokalnie oraz globalnie unikalna. Samo generowanie kodu odbywa się poprzez dodanie do codeManager sygnatury funkcji. Następnie następuje skopiowanie codeManager i wywołanie budowania kodu na startNode. Po zbudowaniu kodu do orginalnego codeManagera zostają dodane w klamrze linie kodu ze skopiowanego codeManagera.

UserFunction
Jest to klasa która dziedziczy po **Function**. Służy ona do obsługi funckji zdefiniowanych przez użytkownika. Przechowuje ona dodatkowo input (ParamsManager) oraz wszyskie referencje nodów, które instancjami typu node danej funkcji. Rozszerza sprawdzenie unikalności zmiennej o parametr. Klasa posiada możliwość zmiany nazwy, oraz typu wyjściowego po takiej zmianie dzięki referencji do nodów oraz returnNodów następuje ich aktualizacja. Również po otrzymaniu informacji o dodaniu/usunięciu parametru zostaje dokonana aktualizacja w nodach poprzez dodanie/usunięcie IO. Generowanie kodu zostało rozszerzone o dodanie do sygnatury parametrów funkcji, oraz po skopiowaniu codeManagera zostają do niego dodane parametry z odpowienim statusem.

UserFunctionManager
Jest to klasa służąca do zarządzania zmiennymi zdefiniowanymi przez użytkownika. Pozwalana ona na dodanie nowej funkcji, oraz usunięcie. Podczas dodawania następuje walidacja unikalności nazwy. Umożliwia ona również znalezienie konkretnej funckji za pomocą PathName.

Templaty
Jest to zbiór klas, przechowujący podstawowe informacje o node takie jak np. nazwa, kategoria, PathName, typ noda. Umożliwiają one również tworzenie nowych instancji nodów.

TempateManager
Jest to klasa odpowiedzialna za stworzenie funkcji wbudowanych oraz wczytanie sygnatór funkcji, klas, metod i konstruktorów zdefiniowanych przez użytkownika w pliku json.

ClassTypeValidator
Jest to klasa typu singleton. Przechowuje ona informacje o dostępnych klasach w programie. Umożliwia dodawanie nowych klas oraz sprawdzenie czy dana nazwa jest klasą.

InstanceCreator
Jest to klasa pomocnicza służąca do tworzenia nowej instancji node za pomocą PathName.

6.2.3 Frontend

W tej warstwie architektury większość klas dziedziczy po klasie zdefiniowanej w bibliotece Unity Engine zwanej Monobehavior. Klasa bazowa definiuje dwie funkcje kluczowe do działania skryptu na scenie, czyli start (ciąg operacji wykonywanych w momencie pojawienia się obiektu na scenie) i update (ciąg operacji wykonywany na każdą klatkę (Frame)). Dodatkowo należy pamiętać, że każda z klas poniżej posiada reprezentację graficzną w postaci Prefab.

ListManager
Jest to klasa bazowa dla kontrolek w postaci listy. Następujące klasy dziedziczą po niej: FunctionListManager, GlobalVariableListManager, LocalVariableListManager, ReferenceListManager. Klasy te rozszarzając List Manager dostarczają odpowiednich funkcjonalności dla elementów UI reprezentujących poszczególne listy.

ButtonScript
Jest to klasa bazowa dla elementów poszczególnych list. Następujące klasy dziedziczą po niej: FunctionButtonScript, InputButtonScript, ReferenceButtonScript, VariableButtonScript. Klasy te udostępniają odpowiednie skrypty dla przycisków dodanych w prefab'ach odpowiednich elementów na listach.

DropDownUI
Skrypt operujący na wbudowanej kontrolce UI silnika Unity. Wypełnia ją odpowiednimi typami dla zmiennych, funkcji i stałych.

NodeBlockManager
Obszerna klasa zarządzająca całą sceną, operacjami na NodeBlock'ach i komunikacją z backendem. Istnieje na scenie tylko jeden taki obiekt. Skrypty używające jej funkcjonalności (np. FunctionListManager zaczytuje listę funkcji użytkownika od właśnie NodeBlockManagera) wyszukują go na scenie za pomocą tag'u.

ViewManager
Klasa pomocnicza dla NodeBlockManagera. Aktywuje i dezaktywuje odpowiednie obiekty Node Bloków na scenie w związku z aktualnym widokiem funkcji zadanym przez użytkownika.

NodeBlockController
Skrypt zarządzający obiektem Node Block. Na początku generuję odpowiednią ilość punktów połączeń i ustawia odpowiednią nazwę. W funkcji Update czeka na zmiany definicji podpiętego pod niego Node Block'a i dynamicznie aktualizuje wygląd na scenie. Dziedziczy po nim NodeBlockControllerInput, który dodaje funkcjonalność pokazywania wartości stałej.

ConnectionPoint
Skrypt odpowiadający za reakcje na naciśnięcie punktu połaczenia i rysowanie graficznej reprezentacji połączenia między Node Block'ami.

Function/Variable/Constant Editor
Skrypty poszczególnnych kontrolek związanych z edycją definicji. Komunikują się z odpowiednimi funkcjami NodeBlockManager, aby poinformować backend o zmianach.

SaveStateModule
Klasa Komunikująca się z NodeBlockManagerem i Backendem, zapisująca stan aplikacji do pliku Json lub ładująca stan z pliku Json.

InfoMessegeManager i InfoMessegeScript
Dwa skrypty implementujące kolejkę wiadomości dla użytkownika od aplikacji Systemu Message i wyświetlanie ich.

SceneRelatedScripts Folder
Znajdują się w nim pliki z klasami odpowiadającymi za takie operacje jak ruch kamerą, czy włączanie ponownie, którejś z kontrolek za pomocą naciśniecia klawisza lub kombinacji klawiszy.

7 Przypadki użycia

7.1 Dodanie funkcji

1. Użytkownik klika w pierwsze pole wpisywalne kontolki Lista Funkcji Użytkownika
2. Wpisuje nazwę funkcji do dodania
3. Naciska przycisk Add

Jeśli funkcja o tej nazwie nie jest już na liście Na liście pojawia się nowy element listy z nazwą nowo dodanej funkcji

W przeciwnym wypadku Pojawia się komunikat z błędem dzięki InfoMessage.

7.2 Edycja funkcji

1. Użytkownik klika w przycisk „Edit" elementu odpowiadającego funkcji którą chce edytować na liście funkcji użytkownika.
2. W edytorze funkcji pojawiają się dane funkcji wybranej przez użytkownika
3. Użytkownik w pole wpisywalne wpisuje nową nazwę dla funkcji

Jeśli podana nowa nazwa jest unikalna Wszystkie Node Block'i funkcji we wszystkich widokach zostaną zaktualizowane z nową nazwą.

W przeciwnym wypadku Pojawia się komunikat z błędem dzięki InfoMessage.

7.3 Dodanie zmiennej

1. Użytkownik klika w pole wpisywalne kontolki Listy globalnych zmiennych
2. Wpisuje nazwę zmiennej do dodania
3. Naciska przycisk Add

Jeśli zmiennej o tej nazwie nie jest już na liście Na liście pojawia się nowy element listy z nazwą nowo dodanej zmiennej

W przeciwnym wypadku Pojawia się komunikat z błędem dzięki InfoMessage.

7.4 Edycja zmiennej

1. Użytkownik klika w przycisk „Edit" elementu odpowiadającego zmiennej którą chce edytować na liście globalnych zmiennych.
2. Pojawia się edytor zmiennych w środkowej częsci okna z danymi zmiennej aktualnie edytowanej
3. Użytkownik w pole wpisywalne wpisuje nową nazwę dla zmiennej
4. Użytkownik wybiera nowy typ zmiennej z kontrolki DropDown
5. Użytkonik naciska przycisk „Save"

Jeśli podana nowa nazwa jest unikalna Wszystkie Node Block'i zmiennej (get i set) we wszystkich widokach zostaną zaktualizowane z nową nazwą i typem.

W przeciwnym wypadku Pojawia się komunikat z błędem dzięki InfoMessage.

7.5 Dodanie input node'a

1. Użytkownik klika w przycisk „Add" elementu z nazwą Input którą znajduje się na liście funkcji wbudowanych.
2. Pojawia się Node Block Input na środku okna w aktualnym widoku funkcji (implementacji funkcji)

7.6 Połącznie bloków

1. Użytkownik klika jeden z punktów połączeń za pomocą lewego przycisku myszy
2. Trzymając przycisk myszy najeżdża kursorem nad drugi punkt połączeń
3. Użytkownik zwalnia przycisk myszy

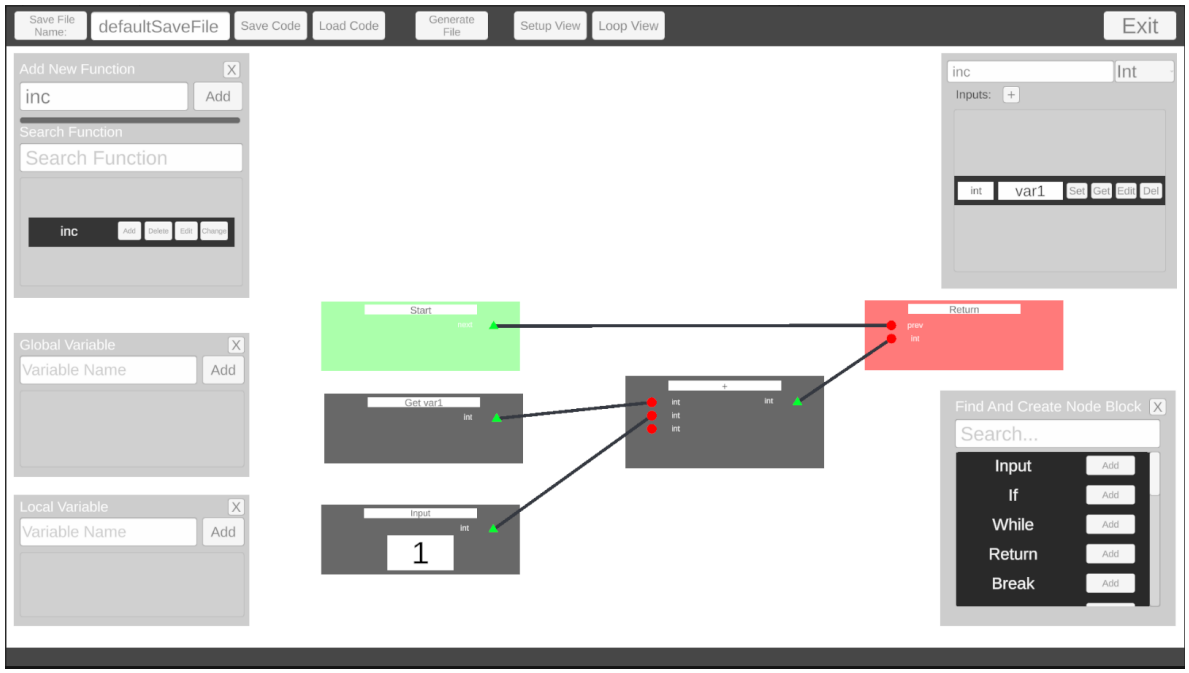
Jeśli typy punktów połączeń odpowiadają sobie w logiczny sposób Node Blocki zostają połączone czarną linią symbolizując połączenie.

W przeciwnym wypadku Pojawia się komunikat z błędem dzięki InfoMessage.

7.7 Prosty funkcja zwiększająca wartość o jeden

1. Użytkownik klika w przycisk „Start View" umieszczony w MenuBarze.
2. Wybiera wyszukuje na liscie funkcji wbudowanych funkcje return i naciska przycisk „Add"
3. Łączy punkt połączeń next Node Block'a Start z punktem połączeń prev Node Block'a Return
4. Użytkownik klika w przycisk „Loop View" umieszczony w MenuBarze.
5. WYbiera wyszukuje na liscie funkcji wbudowanych funkcje return i naciska przycisk „Add"
6. Łączy punkt połączeń next Node Block'a Start z punktem połączeń prev Node Block'a Return
7. W polu wpisywalnym listy funkcji użytkownika wpisuje on „inc" i naciska przycisk „Add"
8. Wyszukuje na liście funkcji użytkonika funkcje „inc" i naciska przycisk „Edit"
9. W edytorze funkcji pojawia się funkcja „inc"
10. Użytkownik naciska przycisk „+" tym samym dodając argument funkcji, następnie na liście pod przyciskiem wyszukuje nowo dodany argumen i naciska przycisk „Get", aby Node Block ze zmienną pojawił się na ekranie

11. Użytkonik wybiera z kontrolki DropDown typ zwracany funkcji jako int
12. Wyszukuje na liście funkcji wbudowanych funkcji Return i „+”, w obu przypadkach naciska przycisk „Add”
13. Użytkownik łączy odpowiednie punkty połączeń Node Block’ów na scenie w logiczny sposób
14. Naciska przycisk „Generate file”



Rysunek 22: Stan Node-Base-Arduino-IDE po powyższych operacjach użytkownika

```
1
2
3   int inc(int var1);
4
5   int inc(int var1)
6   {
7       return var1 + 1;
8   }
9
10  void setup()
11  {
12      return ;
13  }
14
15  void loop()
16  {
17      return ;
18  }
19
```

Rysunek 23: Wygenerowany kod

7.8 Scenariusz zapisu projektu

1. Użytkownik po stworzeniu programu wpisuje w pole wpisywalne w menu nazwę pliku, do którego chce zapisać stan IDE.
2. Klika przycisk menu o nazwie „Save State"
3. Backend waliduje połączenie i definicje Node Blocków tak, aby powstał w miare logiczny kod.

Jeśli przeszedł walidacje Stan IDE zostaje zapisany do pliku Json i pojawia się komunikat dzięki InfoMessage o powodzeniu generacji kodu

W przeciwnym wypadku Pojawia się komunikat z błędem dzięki InfoMessage.

7.9 Scenariusz wczytania projektu

1. Klika przycisk menu o nazwie „Load State"
2. Edytor czyści środowisku usuwając aktualny stan projektu.

Jeśli poprawnie udało się wczytać struktury z pliku json Wczytany zostanie stan poprawnie.

W przeciwnym wypadku Pojawia się komunikat z błędem dzięki InfoMessage.

8 Dodatkowe informacje

8.1 Metryki kodu

8.1.1 Frontend

8.1.2 Backend

9 Podsumowanie

9.1 Wnioski

Celem było stworzenie aplikacji, która umożliwiałaby generowanie kodu arduino za pomocą wizualnych bloczków. Kolejnym celem było zbudowanie przykładowego układu wykorzystującego arduino, a następnie wygenerowanie dla niego kodu przy użyciu aplikacji. Oba cele udało się zrealizować. Aplikacje starano się nieustannie rozszerzać o nowe funkcjonalności.

Aplikacja była rozwijana przez dwuosobowy zespół. Kluczowa była dobra organizacja pracy oraz wymieny informacji, zwłaszcza kiedy trzeba było wprowadzić krytyczne zmiany, które mogły zaburzyć działanie aplikacji. Duży aspekt odegrało też wspólne planowanie interfajsów za pomocą, których odbywała się komunikacja oraz nowych funkcjonalności i oczekiwań co do nich.

Dodatkowe wnioski na temat narzędzi
Bardzo przydatną funkcjonalnością języka okazały się Properties, dzięki którym można było ograniczyć sporą ilość getterów w klasach. Również przydatna w użyciu okazała się, także reflekcja używana jedynie w testach jednostkowych, dzięki niej można było mockować prywatne pola klasy które było tworzone w konstruktorze danej klasy. Natomiast z mniej przyjemnych rzeczy, potrzeba używania virtual jeśli chcemy nadpisywać metodę. Co w połączeniu z używaniem mocków, sprowadzało się do używanie virtual przy każdej metodzie publicznej, którą chcieliśmy zamockować. Alternatywą dla takiego rozwiązanie może być tworzenie klasy dziedziczącej po klasie którą chcemy zamokować, a następnie przesłoniecie metody bazowej za pomocą słowa kluczowego new.

9.2 Usprawnienia

Aplikacje można dalej rozwijać w wielu różnych kierunkach. Poprzez dodawanie nowych funkcjonalności oraz rozwój i modyfikację już zaimplementowanych.

Jednym z takich kierunków może być dalszy rozwój frontendu poprzez pogrupowanie funckji wbudowanych w kategorie jak np. operacje, funckje. Kolejną rzeczą jaką można zrobić jest zwiększenie przejrzystości interfejsu poprzez np. zastosowanie różnych oznaczeń nodow i IO w zależności od typu. Można również umożliwić użytkownikowi wybieranie pliku za pomocą eksploratora plików.

Kolejną funkcjonalnością jaką można dodać jest możliwość dodawnia templatów z poziomu programu zamiast konieczności edytowania pliku json.

Kolejnym kierunkiem rozwoju może być rozbudowa aplikacji poprzez dodawanie kolejnych elementów języka "C++", a nawet po drobnych modyfikacjach wyjście z koncepji generowania kodu dla arduino i zamiast tego generowanie dla czystego C++.

Jednym z takich elementów może być rozwój systemu typów poprzez dodanie typu tablicowego, możliwości definiowania własnych klas przez użytkownika.

Kolejnym elementem może być dodanie instrukcji switch, której zachowanie byłoby podobne do operacji arytmetycznych tzn. dodawałaby nowy input oraz output wraz z podłączeniem ostatniego IO. Wiązałoby się to z wprowadzeniem czegoś podobnego do chainIO tylko dla flowIO oraz kordynacją z chainIO.

Kolejnym kierunkiem rozwoju może być uproszczenie aktualnego kodu, głównie jeśli chodzi o generowanie kodu. Możnaby do tego wykorzystać templaty, które następnie mogłyby posłużyć do uproszczenia generowania nodów.

Literatura

[1] Repozytorium github z projektem: https://github.com/Foxtrot007-ai/Node-Based-Arduino-IDE

[2] Artykuł na wikipedii o Silniku Unity: https://pl.wikipedia.org/wiki/Unity_(silnik_gry)